GATE
CS & DA

Artificial Intelligence

Un-informed Search

Lecture No. 2

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic BFS
- Topic
- Topic
- Topic
- Topic

Topics to be Covered

Topic BFS: theory

Topic DFS: Depth first Search

Topic Compare DFS, BFS.

Topic

Topic

Revision

BFS :

- Queue → FIFO.
- Scan complete level in one go.
- 2 List → Queue, Visited nodes.

Breadth first Search

- Breadth-first search (BFS) is an algorithm for traversing or searching tree or graph data structures.
- It starts at the tree root (or some arbitrary node of a graph, sometimes referred to as a 'search key'), and explores all of the neighbor nodes at the present depth prior to moving on to the nodes at the next depth level.
- It is implemented using a queue (FIFO).
- Breadth First Search (BFS) is a graph traversal algorithm that explores all the vertices in a graph at the current depth before moving on to the vertices at the next depth level. It starts at a specified vertex and visits all its neighbors before moving on to the next level of neighbors. BFS is commonly used in algorithms for pathfinding, connected components, and shortest path problems in graphs.

Breadth first Search

- The Breadth-First Search is a traversing algorithm used to satisfy a given property by searching the tree or graph data structure.
- It belongs to uninformed or blind search AI algorithms as It operates solely based on the connectivity of nodes and doesn't prioritize any particular path over another based on heuristic knowledge or domain-specific information.
- it doesn't incorporate any additional information beyond the structure of the search space. It is optimal for unweighted graphs and is particularly suitable when all actions have the same cost. Due to its systematic search strategy, BFS can efficiently explore even infinite state spaces.

* BFS algo: If the solution exist then BFS will surely find goal state in state space

→ Completeness property of Algo.

Breadth first Search

- Breadth-First Search (BFS) is an algorithm for traversing or searching tree or graph data structures.
- In artificial intelligence (AI), BFS is commonly used for finding the shortest path in **unweighted graphs,** exploring state spaces in search problems, and ensuring all nodes at a given depth are processed before moving deeper.

Breadth first Search

Key Characteristics

- Traversal Method: BFS explores all nodes at the present depth level before moving on to nodes at the next depth level.
- FIFO Structure: Uses a queue (First-In-First-Out) data structure.
- Completeness: BFS is complete, meaning it is guaranteed to find a solution if one exists.

V: S a d b f c i e j G

Q: ~~S a d b f c i e j G~~ K h L

d → f → i

One more example - here one maze ,

	a	b	c	
S	0	0	0	
0	1	1	0	
0	1	0	G	
0	0	0	0	

d, e, f, h, i, j, k, L

S → a → a → b; S → d; d → f; h → G, h → K

BFS

(S a b c e G)

e → G → h, G → L

In this maze:

- S: Starting point
- G: Goal
- 0: An open path
- 1: An obstacle (you can't pass through it)

Breadth first Search

Advantages

- Optimal for Unweighted Graphs: BFS will find the shortest path in terms of the number of edges.
- Complete: Guaranteed to find a solution if one exists.
- Simple Implementation: Straightforward to implement using a queue.

Breadth first Search

Disadvantages

- Memory Usage: Can consume a lot of memory, as it needs to store all nodes at the current depth level. ← inside Queue.
- Not Suitable for Weighted Graphs: BFS does not account for edge weights; Dijkstra's algorithm is more appropriate in such cases.

Breadth first Search

- This algorithm always give a result (if it exists)
- Because before removing the node form the queue we check it with goal node
- And from root to the desired node we can easily follow the parent – parent and get the path from root to the desired node.
- Path Reconstruction : To find the path from A to G, we backtrack using the parent information:

BFS

S → G

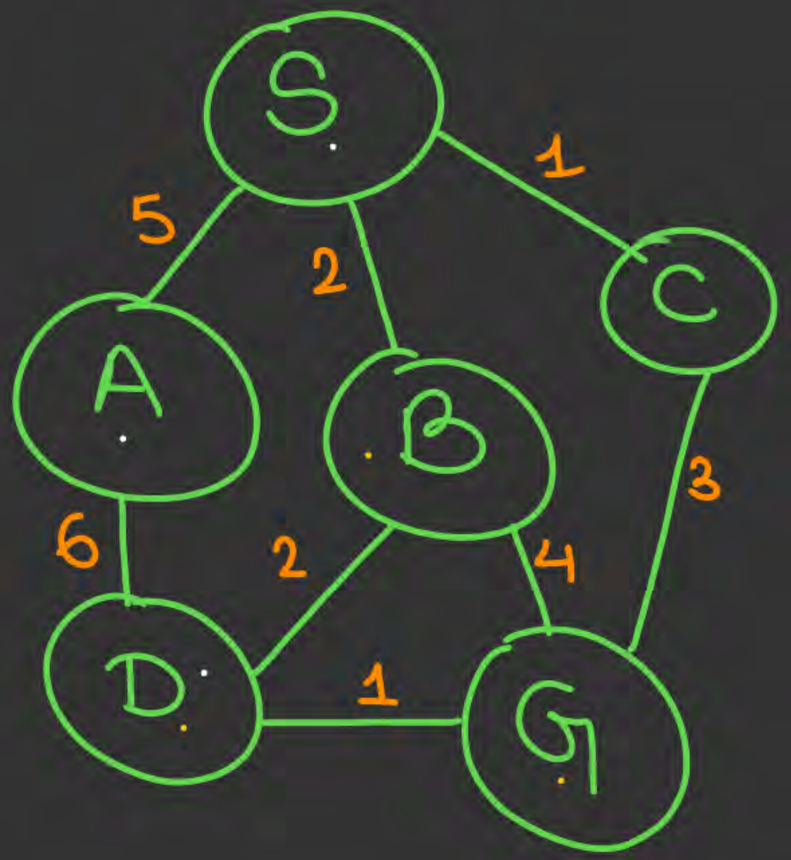
V

S A B C D G end

Q.

~~S~~ ~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~G~~

S B G



S → G

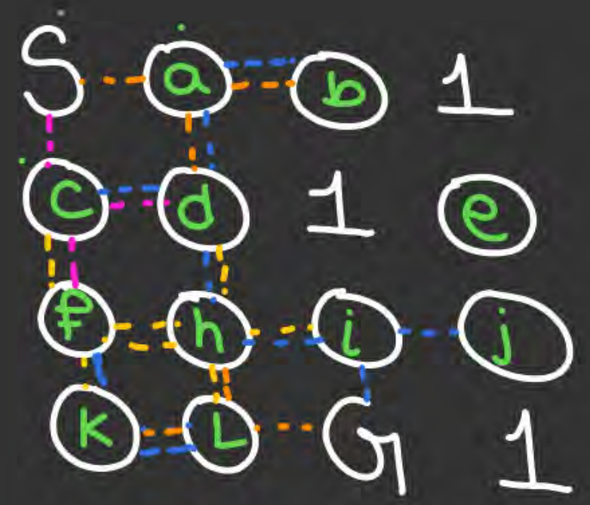
BFS

S A C B D F H K I L G

end.

(S A D H I G) (ans)

Q



Breadth first Search

- This always give the shortest path or the optimal results always.
- Here we don't have weights on the links

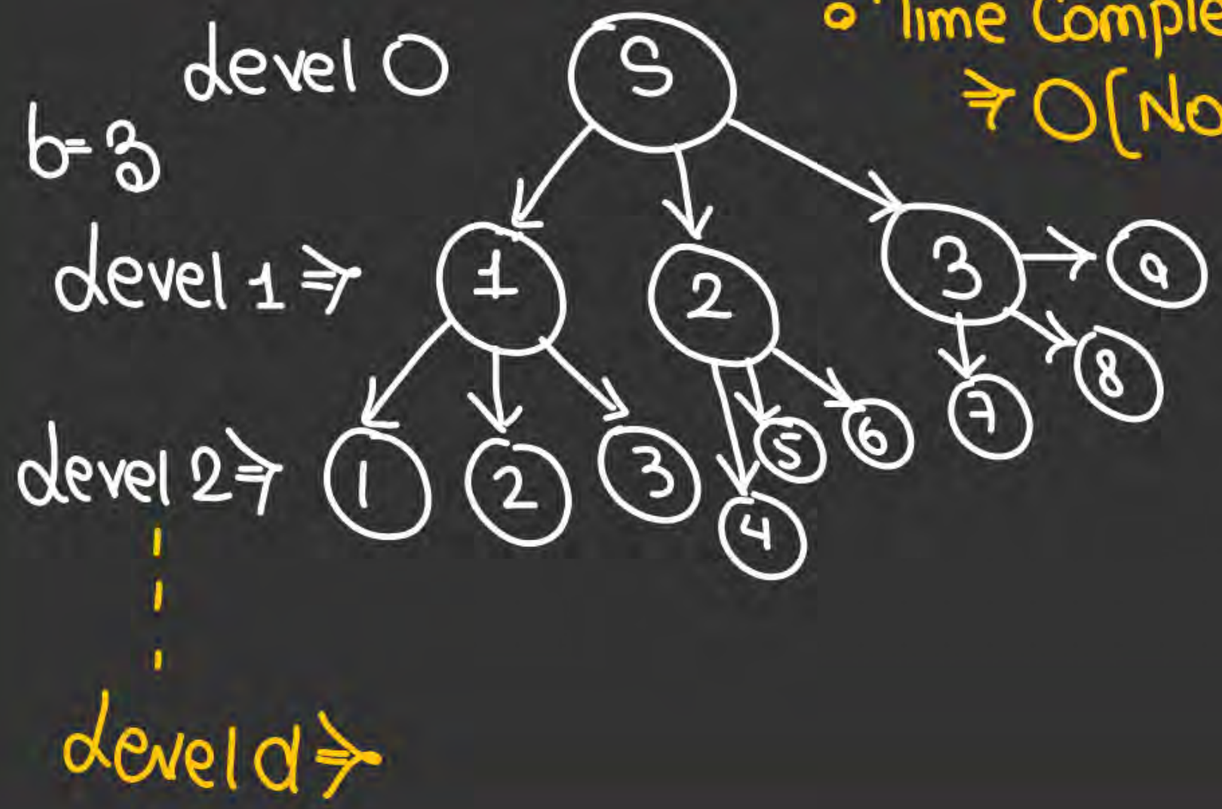
Breadth first Search

- BFS is useful for analyzing the nodes in a graph and constructing the shortest path of traversing through these.
- BFS can traverse through a graph in the smallest number of iterations.
- The architecture of the BFS algorithm is simple and robust.
- The result of the BFS algorithm holds a high level of accuracy in comparison to other algorithms.
- BFS iterations are seamless, and there is no possibility of this algorithm getting caught up in an infinite loop problem.

- Worst Case time Complexity ⇒ BFS
  - Time Complexity ⇒ O[No of nodes]

- d is max depth of graph
- b is max branching factor
  - → One node break into how many node

b = 3



✓ @ level 0: No of node $= 3^0 = 1$

✓ @ " 1: " " " $= 3^1 = 3$

✓ @ " 2: " " " $= 3^2 = 9$

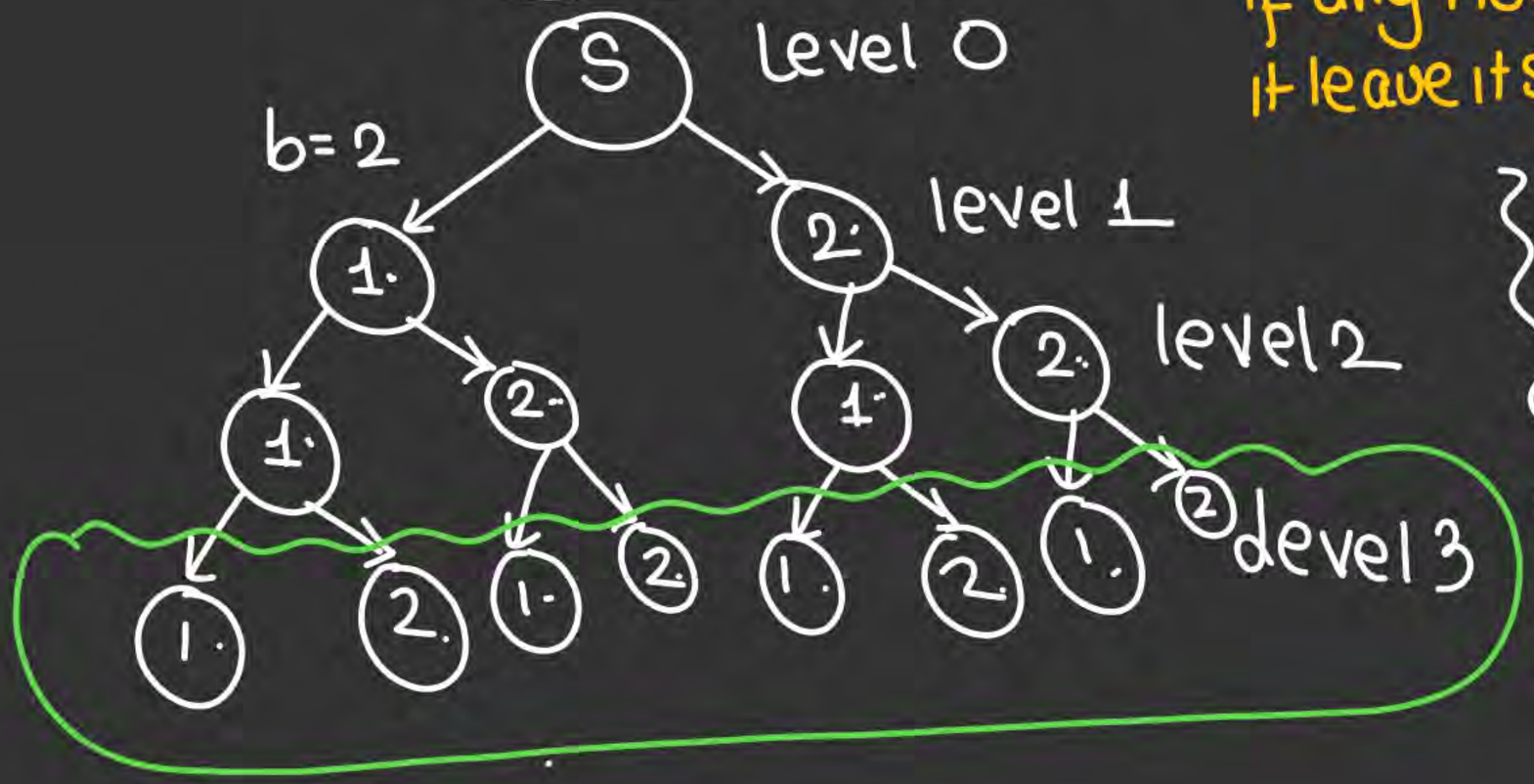
So if b is branching factor, d is no of levels

No of node $\Rightarrow (b^0 + b^1 + b^2 + \ldots b^d) \approx b^d$

$\Rightarrow$ Time Complexity $= O(b^d)$

BFS

Space Complexity



in Queue ⇒
if any node cuts then
it leave its child node

max No of node in Queue
will be 8.

graph

level 0

level 1

branching factor 'b'

level d ⇒ max No of nodes = b^d ✓

So Queue will have to store max b^d No of nodes

⇒ Space Complexity b^d.

BFS

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$$T(b) = 1+b^2+b^3+\ldots\ldots+b^d = O(b^d)$$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

So @ anylevel

No of nodes $\leq b^{\text{No of level}}$

• General

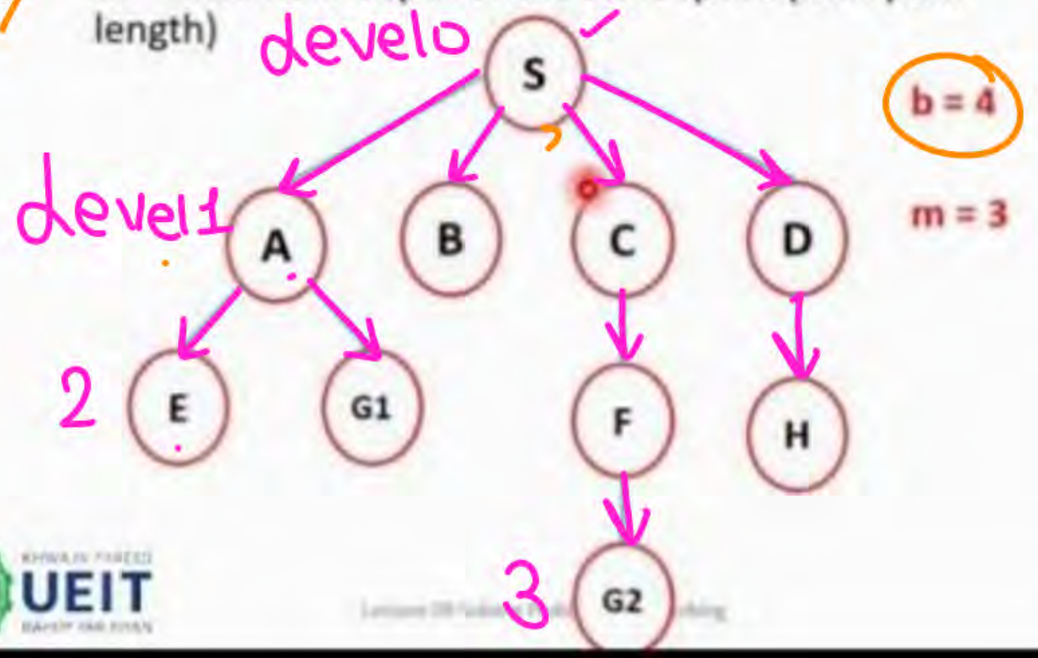
Time Complexity ⇒ $O(b^d)$

BFS

Space Complexity

Comparing Uninformed Search Strategies

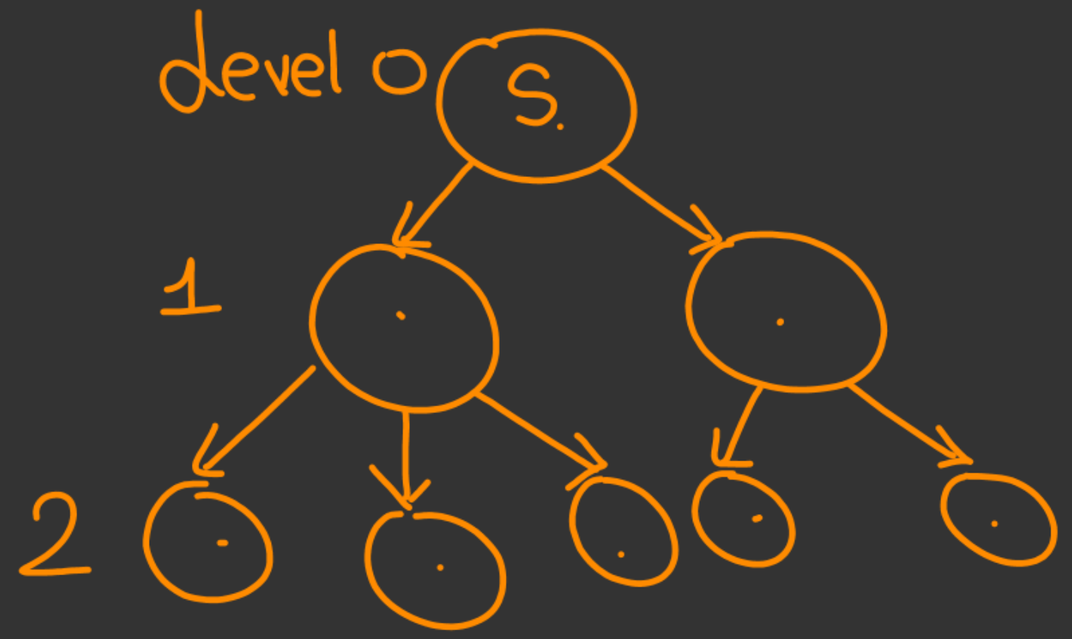
- Time and space complexity are measured in
  - b – maximum branching factor of the search tree
  - m – maximum depth of the state space (Max path length)



UEIT

Q.

depth 2
b = 3



Breadth first Search

- Time Complexity:
- In the worst case, BFS also has a time complexity of $O(B^D)$(Exponential).
- However, BFS systematically explores all nodes at each depth level before moving to the next level.
- It is particularly efficient in finding the shortest path in an unweighted graph because it
- guarantees that the shallowest goal node is reached first.

Breadth first Search

- Space Complexity:
- The space complexity of BFS is typically $O(B^D)$ (Exponential) because it stores ~~all nodes that at each depth level in a queue.~~
- BFS can consume significant memory, especially when the branching factor is high or the tree is deep.
- It guarantees that it explores the shallowest nodes first, but this comes at the cost of increased memory usage.

BFS

algo

Space $
time Complexity

Compleness

FIFO

Optimum path
in nonweighted
graph

disadvantage of BFS

- lot of memory.
- not useful for weighted graph
- Time Complexity is huge.

In place of Queue we use Stack.

Depth first Search

- Depth-First Search (DFS) is a strategy for traversing or searching tree or graph structures.
- In the context of artificial intelligence (AI), DFS is often used for exploring possible states in search problems, finding solutions in game playing, planning, and more.

Depth first Search

- Key Concepts
- Traversal Method: DFS explores as far as possible along each branch before backtracking.
- LIFO Structure: Uses a stack (Last-In-First-Out) data structure, either implicitly through recursion or explicitly.
- Completeness: DFS is not guaranteed to find a solution if one exists, especially in infinite search spaces.

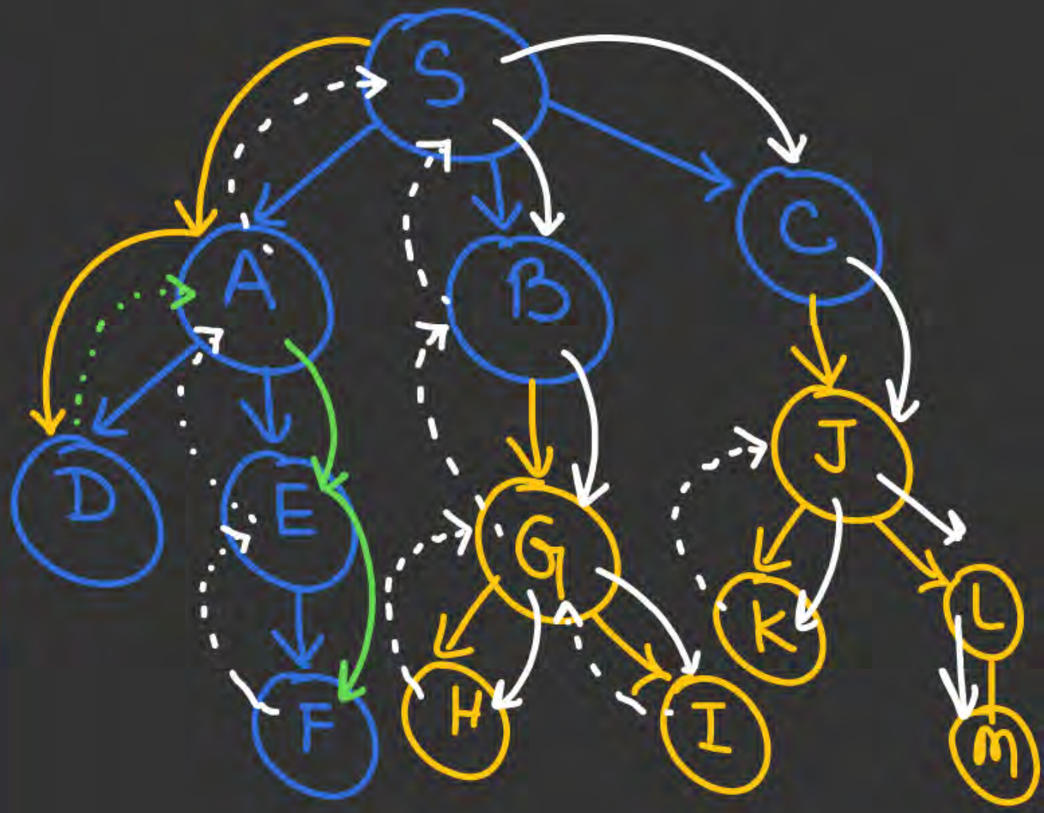
Used for non weighted graph.

Visited

S A D E F B G H I C J K L M

Stack.

~~S~~ ~~C~~ ~~B~~ ~~A~~ ~~E~~ ~~D~~ ~~F~~ ~~G~~ ~~H~~ ~~J~~ ~~L~~ ~~K~~ M



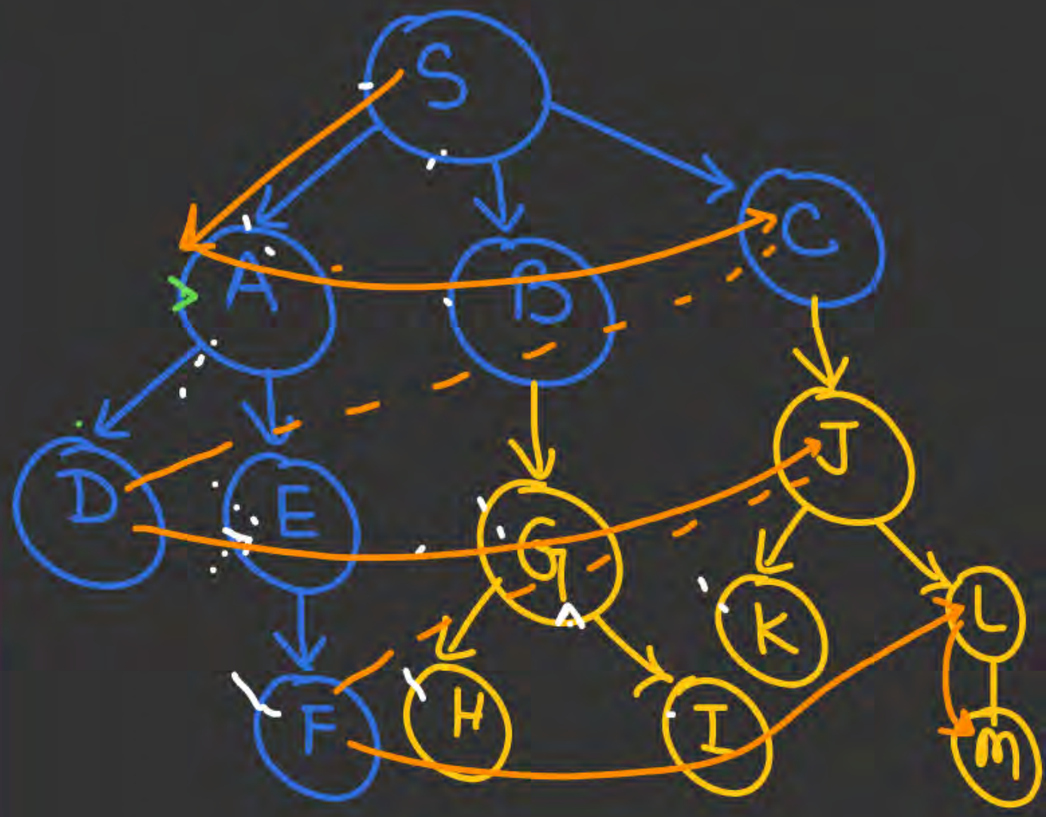
Used for non weighted graph.

Visited

S A D E F B G H I C J K L M

Stack.

~~S C B A E D F G H I J K L M~~



Depth first Search

- Example

- From A to F.

H.W.

```
      2        5
A ------ B ------ D
| \      |      / |
4|  \1   2|    3  |1
|   \    |   /   |
|    \   |  /    |
C ------ E ------ F
      3        1
```

Artificial Intelligence

I and G are the Goal state, solve using BFS and DFS.

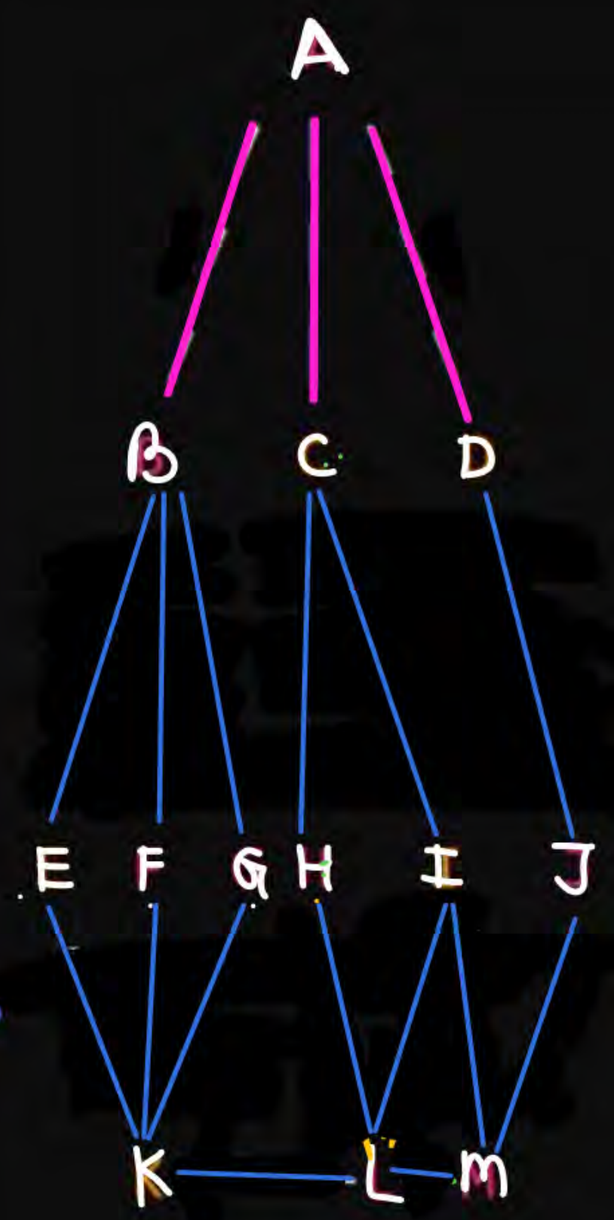
Visited

A B E K L H I goal node.

ABEKLI ✓

Stack

~~A~~ D C ~~B~~ G F ~~E~~ ~~K~~ ~~L~~ m ~~I~~ ~~H~~



BFS → G goal node

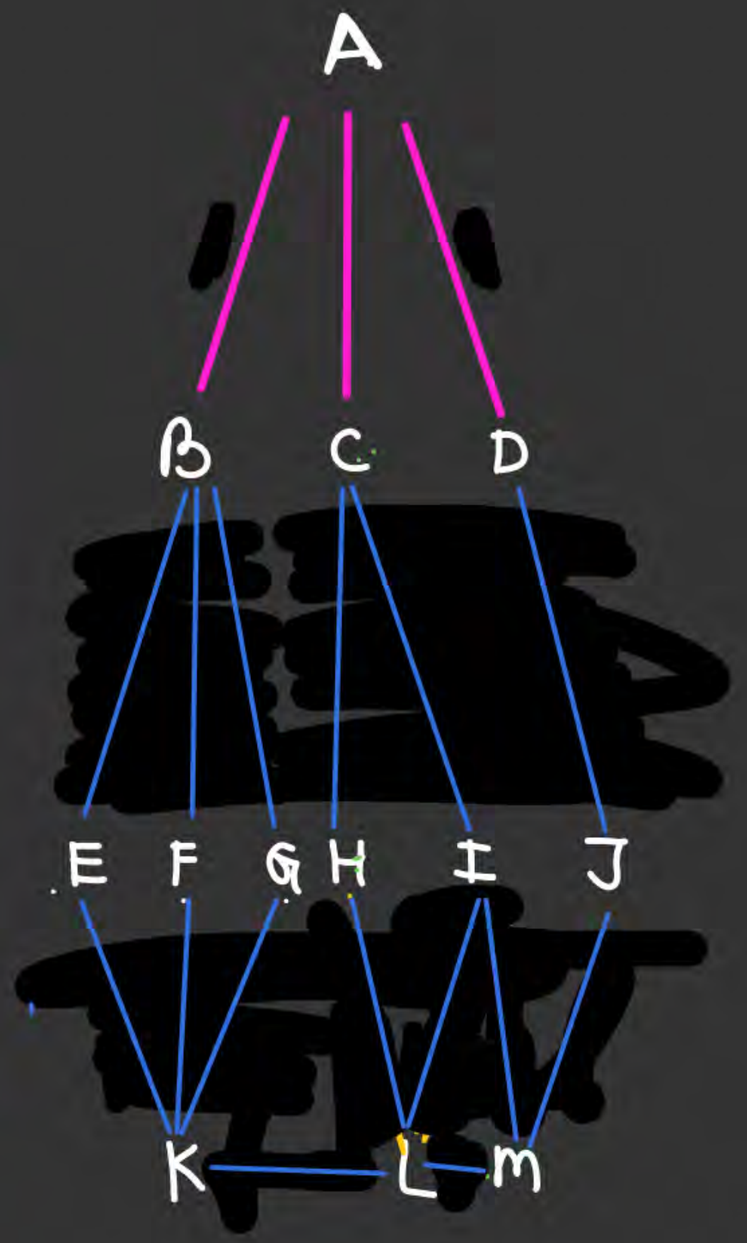
Visited

A B C D E F G Stop

A B G

Queue

~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~E~~ ~~F~~ ~~G~~ H I J K



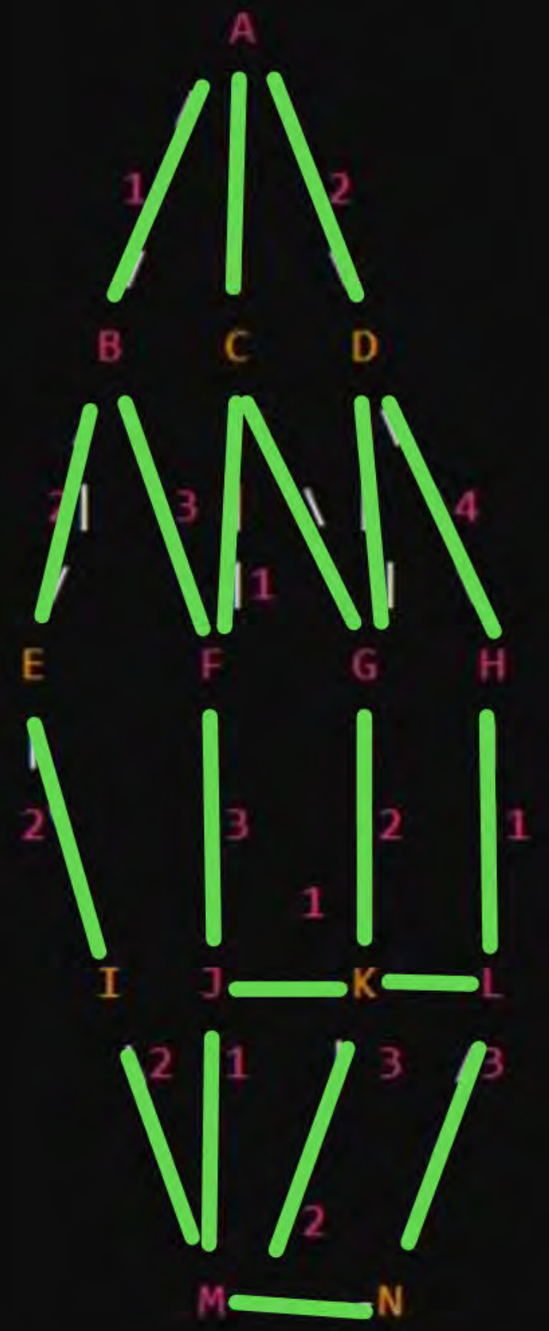
Artificial Intelligence

I and G are the Goal state, solve using BFS and DFS.

Visited

Artificial Intelligence

M and K are the Goal state, solve using BFS and DFS.



Depth first Search

- Time and Space Complexity :

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \ldots\ldots\ldots + n^m = O(n^m)$$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

Depth first Search

Advantage:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

Disadvantage:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

THANK - YOU